

# CS 610 Semester 2024–2025-I: Assignment 3

Name: **Vinayak Devvrat**

Roll Number: **241110079**

October 23, 2024

## NOTE:

All codes attached along with this assignment have been executed on KD first floor lab workstation with ID: **CSEWS179**

## Problem 1

The time taken by the various implementations of the matrix multiplication problem are summarised below. Time values are in ms.

Table 1: Summary of time taken for different implementations of Matrix Multiplication

| SNo. | N    | Sequential | SSE4<br>(Un-aligned) | SSE4<br>(Aligned) | AVX2<br>(Un-aligned) | AVX2<br>(Aligned) | AVX2<br>(V1) | AVX2<br>(V2) |
|------|------|------------|----------------------|-------------------|----------------------|-------------------|--------------|--------------|
| 1    | 256  | 18         | 4                    | 4                 | 1                    | 1                 | 0            | 0            |
| 2    | 512  | 126        | 30                   | 30                | 15                   | 15                | 4            | 3            |
| 3    | 1024 | 1276       | 316                  | 317               | 159                  | 158               | 49           | 31           |
| 4    | 2048 | 9324       | 2614                 | 2593              | 1804                 | 1721              | 395          | 252          |
| 5    | 4096 | 11073      | 2604                 | 2741              | 1678                 | 1772              | 393          | 250          |

- **AVX2 (V1)** is AVX2 implementation along with blocking of  $512 \times 16$  (Chosen through experimentation with different block sizes).
- **AVX2 (V2)** is AVX2 implementation along with blocking of  $512 \times 24$  (Chosen through experimentation with different block sizes).

An unaligned AVX2 implementation with  $512 \times 24$  blocking gives the best performance across different values of N.

**Compilation Command:** `g++ -g -std=c++17 -masm=att -march=native -O2 -fopenmp -fverbose-asm -fno-asynchronous-unwind-tables -fno-exceptions -fno-rtti problem1.cpp -o problem1.out`

## Problem 2

The time taken by the various implementations of the prefix problem are summarised below. Time values are in ms.

Table 2: Summary of time taken for different implementations of Inclusive Prefix Sum

| SNo. | N        | Serial | OMP    | SSE    | AVX2   |
|------|----------|--------|--------|--------|--------|
| 1    | $2^{16}$ | 60     | 40     | 54     | 39     |
| 2    | $2^{30}$ | 495940 | 473691 | 506676 | 460777 |

**Compilation Command:** `g++ -g -std=c++17 -masm=att -march=native -O2 -fopenmp -fverbose-asm -fno-asynchronous-unwind-tables -fno-exceptions -fno-rtti problem2.cpp -o problem2.out`

## Problem 3

Explanations are given as comments alongside the code. Test cases (in `./testcase` directory) and a log file for one of the test cases is attached.

**Compilation Command:** `g++ -g -std=c++17 -masm=att -march=native -O2 -fopenmp -fverbose-asm -fno-asynchronous-unwind-tables -fno-exceptions -fno-rtti problem3.cpp -o problem3.out`

**Run Command:** `./problem3.out file2.txt 15 10 14 output.txt`

## Problem 4

### Part (i)

The following optimizations were made:

- **Common Subexpression Elimination**

There are multiple instances where the same operations are being done many times. Instead of using `xi` values, the actual values are substituted. This reduces repeated arithmetic computations over a long loop nest thereby increasing speed up.

Similarly, multipliers that have to be computed again are also stored in an array before the loop begins.

- **Removing long conditionals**

The big conditional statement where `q1` to `q10` are compared with `e1` to `e10` is removed. Instead negation of each specific condition is checked immediately after the corresponding `q` is calculated. If the condition is not met the entire iteration is skipped using the `continue` keyword.

### Speed Up:

Time taken by code without optimizations = 298.680 seconds

Time taken by code with optimizations = 144.947 seconds

Speed Percentage =  $\left(\frac{298.680 - 144.947}{298.680}\right) \times 100 = \mathbf{51.47\%}$

**Compilation Command:** `gcc -O3 -std=c17 -D_POSIX_C_SOURCE=199309L problem4-v0-p1.c -o prob4-v0.out`

### Part (ii)

Applied the `reduction` and `collapse` directives on the outermost loop on top of the optimizations done in Part (i). Time taken by code without optimizations = 298.680 seconds

Time taken by code with optimizations and OpenMP = 40.255 seconds

Speed Ratio =  $\left(\frac{298.680}{40.255}\right) = \mathbf{7.42}$

**Compilation Command:** `gcc -O3 -std=c17 -D_POSIX_C_SOURCE=199309L problem4-v0-p2.c -o prob4-v0.out -fopenmp`